GestionAgenda

Guide de déploiement : étapes à suivre et configurations à modifier

1. Jenkins et GitHub (CI/CD)
2. Déploiement manuel sur le serveur 151.80.218.41
3. Déploiement avec pipeline CI/CD vers 151.80.218.41 (Jenkins sur le serveur ou sur le PC local)

Version du 1er octobre 2026

Windows Server + IIS · SQL Server · Angular 20 · API .NET 10 · Jenkins · GitHub

Sommaire

1. 0. Hypothèses et architecture cible
2. Configurations à modifier (commun aux trois scénarios)
3. Partie 1 — Jenkins et GitHub (intégration continue)
4. Partie 2 — Déploiement manuel sur le serveur 151.80.218.41
5. Partie 3 — Déploiement avec pipeline CI/CD vers 151
6. Annexe A — Dépannage
7. Annexe B — Liste de contrôle de mise en production
8. Annexe C — Passer en HTTPS avec un nom de domaine

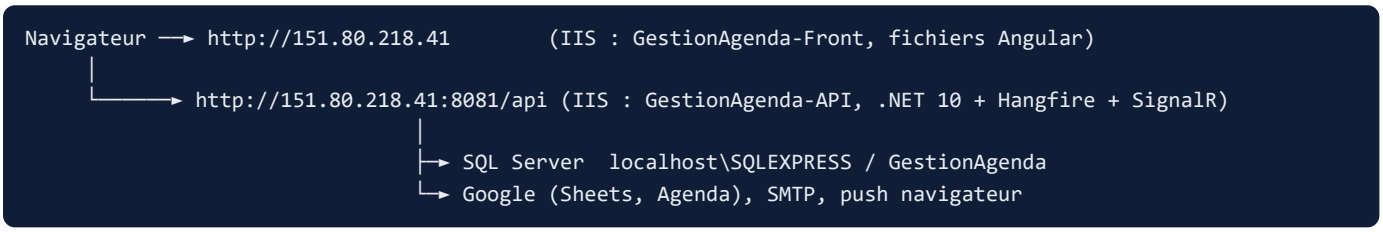
Ce guide couvre trois façons de mettre GestionAgenda en production :

1. **Jenkins et GitHub (CI/CD)** : mettre le code sur GitHub, installer Jenkins, compiler et tester automatiquement à chaque push.
2. **Déploiement manuel** sur le serveur 151.80.218.41 : préparer le serveur une fois, puis installer une version à la main.
3. **Déploiement par pipeline CI/CD** vers 151 : Jenkins installe automatiquement chaque nouvelle version de `master`, que Jenkins tourne **sur le serveur 151** ou **sur votre PC local**.

> **Ordre conseillé.** Lire d'abord la section 0 (architecture) et la section « Configurations à modifier », qui sont communes. La **préparation du serveur** (partie 2, étapes 2.1 à 2.6) est indispensable pour les parties 2 et 3 : on la fait une seule fois. Faites une première mise en service manuelle avant d'automatiser.

0. Hypothèses et architecture cible

Élément	Choix retenu
Serveur	Windows Server 2019/2022, IIS 10, adresse publique `151.80.218.41`
Base de données	SQL Server (Express ou Standard) sur le même serveur, instance `localhost\SQLEXPRESS`, base `GestionAgenda`
Front Angular	Site IIS `GestionAgenda-Front`, port **80**, dossier `C:\GestionAgenda\front`
API .NET 10	Site IIS `GestionAgenda-API`, port **8081**, dossier `C:\GestionAgenda\api`
Appels du navigateur	`http://151.80.218.41:8081/api` (REST) et `http://151.80.218.41:8081/hubs/notifications` (SignalR)
Tâches de fond	Hangfire, hébergé dans l'API (rappels, synchronisation Google Sheets et Google Agenda)



Arborescence sur le serveur

Dossier	Contenu
`C:\GestionAgenda\api`	API publiée + `appsettings.Production.json` (**propre au serveur**, jamais écrasé)
`C:\GestionAgenda\front`	Front Angular + `assets\config.json` (**propre au serveur**, jamais écrasé)
`C:\GestionAgenda\secrets`	Clé JSON du compte de service Google (accès restreint)
`C:\GestionAgenda\livraisons`	Archives `livraison-N.zip` reçues et décompressées
`C:\GestionAgenda\versions\precedente`	Version précédente, pour le retour arrière
`C:\GestionAgenda\sauvegardes`	Sauvegardes `.bak` faites avant chaque migration

> **Nom de domaine et HTTPS.** Le guide utilise l'adresse IP en HTTP pour rester simple. En production réelle, utilisez un nom de domaine et HTTPS (voir annexe C). **Les notifications push du navigateur exigent HTTPS** ; les notifications dans l'application (SignalR), les e-mails et tout le reste fonctionnent en HTTP.

Fichiers fournis dans le dépôt

Fichier	Rôle
`Jenkinsfile`	Pipeline : compilation, tests .NET et Angular, paquet de livraison, déploiement (local ou SSH)
`deploiement/construire-livraison.ps1`	Construit `livraison-N.zip` : API publiée, front de production, script SQL des migrations, `deployer.ps1`, `version.txt`
`deploiement/deployer.ps1`	Exécuté **sur le serveur** : sauvegarde, migrations, arrêt du pool, copie, redémarrage, contrôle de santé
`deploiement/comptes-initiaux.sql`	Modèle SQL pour créer la société, le dirigeant, l'assistante, un collaborateur et une salle
`deploiement/hash-mot-de-passe.cs`	Calcule le hachage BCrypt d'un mot de passe (pour le script SQL)

Configurations à modifier (commun aux trois scénarios)

Ce qui change entre le poste de développement et la production

Réglage	Développement	Production (151)	Où le modifier
Environnement	`Development`	`Production`	Déjà fixé dans `GestionAgenda.API/web.config`
Chaîne de connexion	`localhost\SQLEXPRESS` (votre poste)	Instance SQL du serveur	`appsettings.Production.json` du serveur
`Jwt:Secret`	Clé de dev	**Nouvelle clé aléatoire de 64 caractères**	idem
`Cors:Origins`	`http://localhost:4200`	`http://151.80.218.41` (adresse exacte du front, sans `/` final)	idem
`Seed:Enabled`	`true` (comptes de démo)	`false` (jamais de démo en production)	idem
`Hangfire:Actif`	`true`	`true`	idem
`Notifications:Email`	Vide ou SMTP de test	SMTP réel (hôte, port, compte, mot de passe)	idem
`Notifications:Push`	—	Clés VAPID (HTTPS obligatoire)	idem
Clé Google (`FichierCleCompteService`)	`...\GestionAgenda.API\secrets\...`	`C:\GestionAgenda\secrets\gestion-agenda-compte-service.json`	idem (Sheets et Agenda)
`GoogleCalendar:UrlApplication`	`http://localhost:4200`	`http://151.80.218.41`	idem
`GoogleCalendar:Agendas`	`dirigeant@demo.local` → agenda de test	E-mails réels des dirigeants → leurs agendas Google	idem
`CertificatRacineSupplementaire`	Certificat FortiGate du bureau	**Vide** (sauf proxy qui inspecte le SSL)	idem
`GoogleSheets:FuseauHoraire`	Vide (fuseau du poste)	Fuseau de l'entreprise, ex. `E. Africa Standard Time` ou `Romance Standard Time`	idem
Adresse de l'API côté front	`environment.development.ts` (`http://localhost:5272/api`)	`C:\GestionAgenda\front\assets\config.json`	Fichier du serveur
Swagger	`/swagger`	Désactivé (normal en production)	—
Schéma de la base	Migrations EF en local	Script SQL idempotent `migrations.sql` (inclus dans chaque livraison)	Automatique (`deployer.ps1`)
Comptes utilisateurs	Jeu de démonstration	Script `deploiement/comptes-initiaux.sql`	Une seule fois

> **Fuseau horaire du serveur.** Les serveurs hébergés sont souvent réglés sur UTC. Les rappels et les événements « journée entière » utilisent l'heure locale du serveur. Réglez le serveur sur le fuseau de l'entreprise : `Set-TimeZone -Id "E. Africa Standard Time"` (Antananarivo) ou `Set-TimeZone -Id "Romance Standard Time"` (Paris).

Le fichier `appsettings.Production.json` du serveur

Le fichier `GestionAgenda.API/appsettings.Production.json` versionné dans Git est **un modèle vide** : il ne doit jamais contenir de vraies valeurs. Sur le serveur, on crée `C:\GestionAgenda\api\appsettings.Production.json` avec le contenu ci-dessous. `deployer.ps1` ne l'écrase jamais.

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=localhost\\SQLEXPRESS;Database=GestionAgenda;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Jwt": {
    "Secret": "<64 caractères aléatoires, voir ci-dessous>"
  },
  "Cors": {
    "Origins": [ "http://151.80.218.41" ]
  },
  "Seed": {
    "Enabled": false
  },
  "Hangfire": {
    "Actif": true
  },
  "Notifications": {
    "Email": {
      "Host": "<smtp.fournisseur.com>",
      "Port": 587,
      "Expediteur": "<agenda@votre-domaine>",
      "Utilisateur": "<compte SMTP>",
      "MotDePasse": "<mot de passe SMTP>"
    },
    "Push": {
      "Sujet": "mailto:<agenda@votre-domaine>",
      "ClePublique": "<clé VAPID publique>",
      "ClePrivee": "<clé VAPID privée>"
    }
  },
  "GoogleSheets": {
    "Actif": true,
    "SpreadsheetId": "<identifiant du Google Sheet de production>",
    "Onglet": "Agenda",
    "FichierCleCompteService": "C:\\GestionAgenda\\secrets\\gestion-agenda-compte-service.json",
    "FuseauHoraire": "E. Africa Standard Time",
    "CertificatRacineSupplementaire": ""
  },
  "GoogleCalendar": {
    "Actif": true,
    "FichierCleCompteService": "C:\\GestionAgenda\\secrets\\gestion-agenda-compte-service.json",
    "CertificatRacineSupplementaire": "",
    "UrlApplication": "http://151.80.218.41",
    "LireDisponibilites": true,
    "Agendas": {
      "<email.dirigeant@societe.com>": "<agenda.google.du.dirigeant@gmail.com>"
    }
  }
}
```

Générer le secret JWT (PowerShell, 64 caractères) :

```
$octets = New-Object byte[] 48
[Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($octets)
[Convert]::ToBase64String($octets)
```

Générer les clés VAPID (push navigateur, sur un poste avec Node.js) : `npx web-push generate-vapid-keys`.

Protéger le fichier (lecture seule pour l'API, rien pour les autres) :

```
icacls C:\GestionAgenda\api\appsettings.Production.json /inheritance:r /grant:r "Administrators:F" "SYSTEM:F"
"IIS AppPool\GestionAgenda-API:R"
```

Le fichier `assets\config.json` du front

Le front lit ce fichier au démarrage pour connaître l'adresse de l'API. Sur le serveur, `C:\GestionAgenda\front\assets\config.json` :

```
{
  "apiUrl": "http://151.80.218.41:8081/api"
}
```

`deployer.ps1` ne l'écrase jamais. Le hub SignalR est déduit automatiquement (`http://151.80.218.41:8081/hubs/notifications`).

Ce qui ne doit jamais aller sur GitHub ni dans Jenkins

- La clé du compte de service Google (`secrets/`, `*compte-service*.json`) : elle se copie à la main sur le serveur, dans `C:\GestionAgenda\secrets`.
- `appsettings.Development.json` et toute valeur réelle dans `appsettings.Production.json`.
- Le secret JWT, le mot de passe SMTP, la clé VAPID privée.

Ces règles sont déjà dans le `.gitignore`. Pour la production, **créez une nouvelle clé** pour le compte de service Google (console Google Cloud > IAM > Comptes de service > Clés) au lieu de réutiliser celle du poste de développement, puis supprimez l'ancienne clé quand elle n'est plus utilisée.

Partie 1 — Jenkins et GitHub (intégration continue)

Objectif : à chaque push sur GitHub, Jenkins récupère le code, compile l'API, lance les tests .NET et Angular et produit un paquet `livraison-N.zip`. Le déploiement automatique (partie 3) s'ajoute ensuite sans changer le `Jenkinsfile`.

1.1 Préparer le dépôt local

Le projet n'a pas encore de dépôt distant. Avant le premier push :

1. **Retirer le cache Angular de Git.** Le dossier `GestionAgenda.Front/.angular/cache` a été commité par erreur ; il est maintenant ignoré par `.gitignore`.

```
git rm -r --cached GestionAgenda.Front/.angular
git add .gitignore Jenkinsfile deploiement docs
git commit -m "Préparation du déploiement : Jenkinsfile, scripts, guide"
```

2. **Vérifier qu'aucun secret n'est suivi**, ni aujourd'hui ni dans l'historique (chaque commande doit ne rien afficher) :

```
git ls-files | Select-String 'secrets/|compte-service|appsettings.Development'
git log --all --name-only --format= | Select-String 'secrets/|compte-service|appsettings.Development' |
Sort-Object -Unique
```

   Si la seconde commande affiche un fichier, l'historique contient un secret : ne poussez pas, nettoyez l'historique (outil `git filter-repo`) et **révoquez la clé concernée**.

1.2 Créer le dépôt GitHub

1. Sur GitHub : **New repository**, nom `GestionAgenda`, visibilité **Private**, sans README ni `.gitignore`.
2. Pousser le code :

```
git remote add origin https://github.com/<organisation>/GestionAgenda.git
git push -u origin master
```

3. **Protéger** `master` (Settings > Branches > Add rule) : exiger une pull request et le statut Jenkins vert avant fusion (après le premier build Jenkins, le statut apparaît dans la liste).
4. **Jeton d'accès pour Jenkins** : GitHub > Settings > Developer settings > *Fine-grained tokens* > accès au seul dépôt `GestionAgenda`, permissions *Contents : Read*, *Metadata : Read*, *Commit statuses : Read and write*, *Pull requests : Read*. Notez le jeton : il ira dans les identifiants Jenkins.

1.3 Installer Jenkins — deux cas possibles

	Cas A : Jenkins **sur le serveur 151**	Cas B : Jenkins **sur votre PC local**
Déclenchement	Webhook GitHub (immédiat)	Scrutation périodique (GitHub ne peut pas joindre votre PC)
Déploiement	Copie locale sur le serveur	Copie SSH vers 151
Avantage	Toujours disponible, rapide	Rien à installer de lourd sur le serveur de production
Contrainte	Consomme de la mémoire (prévoir 4 Go de plus) ; port 8080 à protéger	Le PC doit être allumé ; port 22 sortant autorisé par le pare-feu du bureau

Installation (identique dans les deux cas, sur la machine Jenkins)

1. **Java 21** (Eclipse Temurin JDK 21, installeur MSI), cocher « Set JAVA_HOME ».
2. **Jenkins LTS** pour Windows (installeur MSI depuis jenkins.io), port **8080**.

- Compte du service : créez d'abord un compte local `svc-jenkins` (mot de passe fort, n'expire pas), et donnez-lui le droit « Ouvrir une session en tant que service ». Choisissez-le dans l'installeur (*Run service as local or domain user*).
- **Cas A** : `svc-jenkins` doit être **administrateur local** du serveur (arrêt du pool IIS, écriture dans `C:\GestionAgenda`).
- **Cas B** : un compte standard suffit.

3. Ouvrir `http://localhost:8080`, saisir le mot de passe initial (`C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword`), installer les plugins suggérés, créer le compte administrateur.
4. **Outils à installer sur la machine Jenkins** (dans le PATH du compte `svc-jenkins`) :
   - Git for Windows ;
   - **SDK .NET 10** (le SDK, pas seulement le runtime) ;
   - **Node.js 22 LTS ou 24 LTS** (Angular 20 n'accepte pas les versions plus anciennes) ;
   - Cas A : `sqlcmd` (fourni avec SQL Server, ou `winget install sqlcmd`) ;
   - Cas B : client OpenSSH (intégré à Windows 10/11 : `ssh -V` doit répondre).
5. Redémarrer le service Jenkins après l'installation des outils, pour qu'il voie le nouveau PATH.

1.4 Plugins Jenkins

Administrer Jenkins > Plugins > Available, installer : **Pipeline**, **Git**, **GitHub**, **GitHub Branch Source**, **Credentials Binding**, **SSH Credentials**, **Timestamper**, **Workspace Cleanup**. (La plupart font partie des plugins suggérés.)

1.5 Identifiants Jenkins

Administrer Jenkins > Credentials > System > Global credentials > **Add Credentials** :

ID	Type	Contenu
`github-ga`	Username with password	Utilisateur GitHub + **jeton** de l'étape 1.2 comme mot de passe
`ga-test-sql`	Secret text	Chaîne de connexion d'une base **de test** : `Server=localhost\SQLEXPRESS;Database=GestionAgenda_Integration;Trusted_Connection=True;TrustServerCertificate=True`
`srv151-ssh`	SSH Username with private key	Cas B uniquement, voir partie 3

> **Attention : `ga-test-sql` ne doit jamais pointer sur la base de production.** Le test `MigrationDemandesTests` applique les migrations sur la base indiquée. Sur la machine Jenkins, donnez au compte `svc-jenkins` le droit de créer cette base de test : `sqlcmd -S localhost\SQLEXPRESS -E -Q "CREATE LOGIN [<MACHINE>\svc-jenkins] FROM WINDOWS; ALTER SERVER ROLE dbcreator ADD MEMBER [<MACHINE>\svc-jenkins];"` S'il n'y a pas de SQL Server sur la machine Jenkins, remplacez la commande de test du `Jenkinsfile` par : `dotnet test GestionAgenda.Tests -c Release --no-build --filter "FullyQualifiedName!~MigrationDemandesTests" ...`

1.6 Le `Jenkinsfile`

Le `Jenkinsfile` à la racine du dépôt contient les étapes suivantes :

Étape	Commandes	Résultat
API : compilation	`dotnet restore` puis `dotnet build GestionAgenda.slnx -c Release`	Compilation de toute la solution
API : tests	`dotnet test GestionAgenda.Tests` (environ 106 tests)	Résultats `.trx` archivés
Front : dépendances et tests	`npm ci`, `npx ng test --watch=false` (environ 32 tests)	Tests Angular (vitest)
Paquet de livraison	`deploiement\construire-livraison.ps1`	`livraison-N.zip` archivé dans Jenkins
Déploiement sur 151	`deployer.ps1` (local ou par SSH)	**Seulement sur** `master` et si `MODE_DEPLOIEMENT` est défini (partie 3)

Tant que la variable `MODE_DEPLOIEMENT` n'est pas définie dans Jenkins, le pipeline fait uniquement l'intégration continue : rien n'est déployé.

1.7 Créer le job

1. Tableau de bord > **New Item** > nom `GestionAgenda` > **Multibranch Pipeline** (obligatoire : la condition « seulement `master` » du `Jenkinsfile` ne fonctionne qu'avec ce type de job).
2. *Branch Sources* > Add source > **GitHub** : identifiants `github-ga`, URL `https://github.com/<organisation>/GestionAgenda.git`.
3. *Build Configuration* : by Jenkinsfile, chemin `Jenkinsfile`.
4. *Scan Multibranch Pipeline Triggers* :
   - Cas A : laisser décoché (le webhook déclenche) ou mettre 1 jour en secours.
   - Cas B : cocher **Periodically if not otherwise run**, intervalle **2 minutes** ou **5 minutes**.
5. Enregistrer : Jenkins scanne le dépôt et lance un premier build sur `master`.

1.8 Déclenchement automatique

- **Cas A (Jenkins sur 151)** : GitHub > dépôt > Settings > Webhooks > Add webhook :
  - Payload URL : `http://151.80.218.41:8080/github-webhook/` (avec le `/` final)
  - Content type : `application/json`
  - Événements : *Pushes* et *Pull requests*
  - Ouvrir le port 8080 **uniquement** pour votre bureau et les adresses des webhooks GitHub (liste « hooks » sur `https://api.github.com/meta`) :

    ```
    New-NetFirewallRule -DisplayName "Jenkins" -Direction Inbound -Protocol TCP -LocalPort 8080 -Action
    Allow -RemoteAddress <IP du bureau>,<plages hooks GitHub>
    ```

- **Cas B (Jenkins sur le PC)** : la scrutation de l'étape 1.7 suffit. Variante : relais de webhook (smee.io) si l'on veut un déclenchement immédiat.

1.9 Vérifier

Faites un petit commit sur une branche, ouvrez une pull request : Jenkins doit lancer le build et afficher le statut dans la pull request. Sur `master`, l'artefact `livraison-N.zip` doit apparaître dans la page du build.

Partie 2 — Déploiement manuel sur le serveur 151.80.218.41

Les étapes 2.1 à 2.6 préparent le serveur ; elles se font **une seule fois**. Les étapes 2.7 à 2.10 installent une version. Toutes les commandes se lancent dans **PowerShell en administrateur**, sur le serveur (connexion Bureau à distance).

2.1 Rôles et composants Windows

```
Install-WindowsFeature Web-Server, Web-WebSockets, Web-AppInit, Web-Http-Redirect, Web-Mgmt-Console -
IncludeManagementTools
Set-TimeZone -Id "E. Africa Standard Time"   # adapter au fuseau de l'entreprise
```

- `Web-WebSockets` : nécessaire aux notifications en temps réel (SignalR).
- `Web-AppInit` : permet de garder l'API toujours démarrée (Hangfire doit tourner en continu).

Puis installer, dans cet ordre :

1. **ASP.NET Core 10.0 Runtime — Windows Hosting Bundle** (page de téléchargement .NET 10 de Microsoft). Ensuite : `net stop was /y` puis `net start w3svc`.
2. **IIS URL Rewrite 2.1** (iis.net) : utilisé par le `web.config` du front pour les routes Angular.
3. **SQL Server 2022** (Express suffit pour démarrer) : instance `SQLEXPRESS`, authentification Windows, cocher *Ajouter l'utilisateur actuel* comme administrateur SQL. Vérifier que `sqlcmd` répond : `sqlcmd -S localhost\SQLEXPRESS -E -Q "SELECT @@VERSION"`.

2.2 Dossiers et droits

```
'api','api\logs','front','secrets','livraisons','versions','sauvegardes' |
    ForEach-Object { New-Item -ItemType Directory -Force "C:\GestionAgenda\$_" | Out-Null }
```

Les droits NTFS se posent à la fin de l'étape 2.3, car les identités des pools n'existent qu'une fois les pools créés.

2.3 Sites et pools IIS

```
Import-Module WebAdministration

# Le site par défaut occupe le port 80
Stop-Website 'Default Web Site'
Set-ItemProperty 'IIS:\Sites\Default Web Site' serverAutoStart False

# API : pas de code managé .NET Framework, toujours démarrée, jamais recyclée par inactivité
New-WebAppPool GestionAgenda-API
Set-ItemProperty IIS:\AppPools\GestionAgenda-API managedRuntimeVersion ''
Set-ItemProperty IIS:\AppPools\GestionAgenda-API startMode AlwaysRunning
Set-ItemProperty IIS:\AppPools\GestionAgenda-API processModel.idleTimeout ([TimeSpan]::Zero)
Set-ItemProperty IIS:\AppPools\GestionAgenda-API recycling.periodicRestart.time ([TimeSpan]::Zero)
New-Website -Name GestionAgenda-API -Port 8081 -PhysicalPath C:\GestionAgenda\api -ApplicationPool
GestionAgenda-API
Set-ItemProperty IIS:\Sites\GestionAgenda-API applicationDefaults.preloadEnabled $true

# Front : fichiers statiques
New-WebAppPool GestionAgenda-Front
Set-ItemProperty IIS:\AppPools\GestionAgenda-Front managedRuntimeVersion ''
New-Website -Name GestionAgenda-Front -Port 80 -PhysicalPath C:\GestionAgenda\front -ApplicationPool
GestionAgenda-Front

# Droits NTFS
icacls C:\GestionAgenda\api /grant "IIS AppPool\GestionAgenda-API:(OI)(CI)RX"
icacls C:\GestionAgenda\api\logs /grant "IIS AppPool\GestionAgenda-API:(OI)(CI)M"
icacls C:\GestionAgenda\front /grant "IIS AppPool\GestionAgenda-Front:(OI)(CI)RX" "IUSR:(OI)(CI)RX"
icacls C:\GestionAgenda\secrets /inheritance:r /grant:r "Administrators:(OI)(CI)F" "SYSTEM:(OI)(CI)F" "IIS
AppPool\GestionAgenda-API:(OI)(CI)R"
icacls C:\GestionAgenda\sauvegardes /grant "NT Service\MSSQL`$SQLEXPRESS:(OI)(CI)M"
```

> Pourquoi `AlwaysRunning` et `preloadEnabled` ? Par défaut, IIS arrête une application après 20 minutes sans visite. Hangfire s'arrêterait aussi : plus de rappels ni de synchronisation Google. Ces réglages gardent l'API démarrée en permanence.

2.4 Base de données

```
sqlcmd -S localhost\SQLEXPRESS -E -b -Q "CREATE DATABASE [GestionAgenda]"
sqlcmd -S localhost\SQLEXPRESS -E -b -Q "CREATE LOGIN [IIS APPPOOL\GestionAgenda-API] FROM WINDOWS; USE
[GestionAgenda]; CREATE USER [IIS APPPOOL\GestionAgenda-API] FOR LOGIN [IIS APPPOOL\GestionAgenda-API]; ALTER
ROLE db_owner ADD MEMBER [IIS APPPOOL\GestionAgenda-API];"
```

- L'API se connecte avec l'identité de son pool IIS : **aucun mot de passe** dans la chaîne de connexion.
- `db_owner` est nécessaire car Hangfire crée son propre schéma (`HangFire`) au premier démarrage.
- Le compte qui exécute `deployer.ps1` (vous, ou `svc-jenkins`, ou le compte SSH de déploiement) doit aussi pouvoir sauvegarder et migrer la base : administrateur SQL, ou `db_owner` sur `GestionAgenda`.
- Ne jamais ouvrir le port SQL 1433 vers Internet.

2.5 Pare-feu

```
New-NetFirewallRule -DisplayName "GestionAgenda Front" -Direction Inbound -Protocol TCP -LocalPort 80,443 -
Action Allow
New-NetFirewallRule -DisplayName "GestionAgenda API"   -Direction Inbound -Protocol TCP -LocalPort 8081 -
Action Allow
```

Si l'hébergeur propose un pare-feu réseau (panneau d'administration), y autoriser aussi 80, 443 et 8081. Le serveur doit pouvoir **sortir** vers `*.googleapis.com` (443), le serveur SMTP (587) et les services de push (`fcm.googleapis.com`, etc.).

2.6 Configuration propre au serveur

1. Créer `C:\GestionAgenda\api\appsettings.Production.json` (modèle dans « Configurations à modifier ») et le protéger avec `icacls`.
2. Créer `C:\GestionAgenda\front\assets\config.json` avec `{"apiUrl": "http://151.80.218.41:8081/api"}`.
3. Copier la clé Google dans `C:\GestionAgenda\secrets\gestion-agenda-compte-service.json` (par le Bureau à distance, jamais par Git).
4. Google Agenda : chaque dirigeant partage son agenda avec le compte de service, avec le droit « Apporter des modifications aux événements » (voir `docs/google-agenda.md`). Google Sheets : partager le Sheet de production avec le compte de service en « Éditeur » (voir `docs/google-sheets.md`).

2.7 Construire la livraison (sur le poste de développement)

```
cd D:\softwell_travail\GestionAgenda\GestionAgenda
git pull
powershell -ExecutionPolicy Bypass -File deploiement\construire-livraison.ps1 -Sortie
"$env:USERPROFILE\Desktop\livraison-manuelle-$(Get-Date -Format yyyyMMdd-HHmm).zip"
```

Le script publie l'API en Release, construit le front en production, génère le script SQL idempotent `migrations.sql` et ajoute `deployer.ps1` et `version.txt` (numéro de commit) dans l'archive.

Lancez les tests avant de livrer :

```
dotnet test GestionAgenda.Tests
cd GestionAgenda.Front; npx ng test --watch=false
```

2.8 Installer la livraison sur le serveur

1. Copier l'archive dans `C:\GestionAgenda\livraisons\` (copier-coller par le Bureau à distance).
2. Extraire `deployer.ps1` de l'archive (ou le prendre dans le dépôt) et le placer dans `C:\GestionAgenda\livraisons\`.
3. Lancer, en PowerShell administrateur :

   ```
   powershell -ExecutionPolicy Bypass -File C:\GestionAgenda\livraisons\deployer.ps1 -Archive
   C:\GestionAgenda\livraisons\livraison-manuelle-20261001-1000.zip
   ```

Le script enchaîne : décompression et contrôle du contenu → **sauvegarde de la base** → **migrations** → arrêt du pool de l'API → copie de la version actuelle dans `versions\precedente` → copie de la nouvelle API et du nouveau front (sans toucher `appsettings.Production.json` ni `config.json`) → redémarrage → **contrôle de santé** (l'API doit répondre `401` sur `/api/demandes-rencontre`, preuve qu'elle a démarré). Au moindre problème, il s'arrête avec un message.

Paramètres modifiables si votre serveur diffère : `-InstanceSql`, `-Base`, `-Racine`, `-PoolApi`, `-UrlSante`.

2.9 Créer les premiers comptes (première installation uniquement)

En production, le jeu de démonstration n'est jamais chargé et l'application n'a pas d'écran d'administration des comptes. On crée donc les premiers utilisateurs en SQL.

1. Sur le poste de développement, calculer le hachage de chaque mot de passe :

   ```
   dotnet run deploiement/hash-mot-de-passe.cs -- "MotDePasseFort!2026"
   ```

Résultat : une chaîne de 60 caractères commençant par `$2a$11$`.

2. Copier `deploiement/comptes-initiaux.sql`, remplacer toutes les valeurs `<...>` (société, noms, e-mails, hachages, salle).
3. Sur le serveur :

```
sqlcmd -S localhost\SQLEXPRESS -E -b -I -d GestionAgenda -i C:\GestionAgenda\livraisons\comptes-initiaux.sql
```

4. Supprimer ensuite le fichier SQL rempli (il contient les e-mails et les hachages).

Le script crée la société, un dirigeant, son assistante (avec l'affectation), un collaborateur et une salle ; dupliquez les lignes pour d'autres personnes. Rôles possibles : `Dirigeant`, `Assistante`, `Collaborateur`.

2.10 Vérifications après installation

Contrôle	Résultat attendu
`http://151.80.218.41`	Page de connexion GestionAgenda
Connexion avec un compte créé	Accès à l'application, sans erreur CORS dans la console du navigateur (F12)
Créer une demande, l'accepter, la planifier	Notification en temps réel chez l'autre utilisateur
Agenda > panneau Google Agenda > **Vérifier l'accès**	« Accès en écriture » pour chaque dirigeant configuré
Google Sheet	La demande apparaît dans le Sheet de production en moins d'une minute
Observateur d'événements > Application	Aucune erreur « IIS AspNetCore Module »

2.11 Mises à jour suivantes et retour arrière

Mise à jour : refaire seulement 2.7 et 2.8.

Retour arrière (si la nouvelle version pose problème) :

```
Import-Module WebAdministration
Stop-WebAppPool GestionAgenda-API
robocopy C:\GestionAgenda\versions\precedente\api   C:\GestionAgenda\api   /MIR /XD logs /XF appsettings.Production.json
robocopy C:\GestionAgenda\versions\precedente\front C:\GestionAgenda\front /MIR /XF config.json
Start-WebAppPool GestionAgenda-API
```

Si les migrations de la nouvelle version n'ont fait qu'ajouter des tables ou des colonnes (cas le plus courant), l'ancienne version fonctionne avec la base migrée. Sinon, restaurer la sauvegarde faite juste avant (`C:\GestionAgenda\sauvegardes\GestionAgenda-<date>.bak`) depuis SQL Server Management Studio, pool de l'API arrêté.

Partie 3 — Déploiement avec pipeline CI/CD vers 151

Objectif : chaque fusion sur `master` est testée puis installée automatiquement sur 151.80.218.41, sans intervention.
Prérequis : parties 1 et 2 (2.1 à 2.6 et au moins une installation manuelle réussie, comptes créés).

3.1 Principe

Le même `Jenkinsfile` sert aux deux cas. Seule la variable globale Jenkins `MODE_DEPLOIEMENT` change :

```
push sur master → Jenkins : compilation → tests .NET → tests Angular → livraison-N.zip
                                                                              |
                          MODE_DEPLOIEMENT = local (Jenkins sur 151) ─────────┤→ deployer.ps1 en local
                          MODE_DEPLOIEMENT = ssh   (Jenkins sur le PC) ──────┘→ scp + ssh → deployer.ps1 sur
151
                                                                              |
                                                                vérification : http://151.80.218.41 répond 200
```

Si une étape échoue (un test, une migration, le contrôle de santé), le pipeline s'arrête et l'ancienne version reste en place ou se restaure avec la procédure 2.11.

3.2 Cas A — Jenkins installé sur le serveur 151

1. Jenkins est installé selon la partie 1 (cas A), service exécuté par `svc-jenkins`, **administrateur local**.
2. Donner à `svc-jenkins` les droits SQL pour sauvegarder et migrer la base de production :

   ```
   sqlcmd -S localhost\SQLEXPRESS -E -b -Q "CREATE LOGIN [<NOM-SERVEUR>\svc-jenkins] FROM WINDOWS; USE
   [GestionAgenda]; CREATE USER [<NOM-SERVEUR>\svc-jenkins] FOR LOGIN [<NOM-SERVEUR>\svc-jenkins]; ALTER ROLE
   db_owner ADD MEMBER [<NOM-SERVEUR>\svc-jenkins];"
   ```

   (Le même login reçoit `dbcreator` pour la base de test, voir 1.5.)
3. Administrer Jenkins > System > **Global properties** > Environment variables :

Nom	Valeur
`MODE_DEPLOIEMENT`	`local`
`URL_PUBLIQUE`	`http://151.80.218.41`

4. Lancer un build de `master` : l'étape « Déploiement sur 151.80.218.41 » copie l'archive dans `C:\GestionAgenda\livraisons` et exécute `deployer.ps1`.

> Sécurité : Jenkins sur le serveur de production expose une interface d'administration. Mot de passe administrateur fort, port 8080 filtré (étape 1.8), inscriptions désactivées, plugins tenus à jour.

3.3 Cas B — Jenkins sur votre PC local, déploiement par SSH

Sur le serveur 151 (une fois) :

1. Installer et démarrer le serveur OpenSSH :

   ```
   Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0
   Set-Service sshd -StartupType Automatic
   Start-Service sshd
   ```
2. Créer le compte de déploiement `deploy-ga` (administrateur local : il arrête le pool et écrit dans `C:\GestionAgenda`) et lui donner les droits SQL comme en 3.2, étape 2 (avec `<NOM-SERVEUR>\deploy-ga`).
3. Restreindre SSH à l'adresse publique du bureau (remplacer la règle créée par l'installation) :

```
Get-NetFirewallRule -Name *OpenSSH* | Remove-NetFirewallRule
New-NetFirewallRule -DisplayName "SSH déploiement" -Direction Inbound -Protocol TCP -LocalPort 22 -Action
Allow -RemoteAddress <IP publique du bureau>
```

Sur le PC Jenkins :

4. Générer une paire de clés dédiée (sans phrase secrète, pour l'automatisation) :

```
ssh-keygen -t ed25519 -f $env:USERPROFILE\.ssh\deploy-ga-151 -C "jenkins-gestionagenda" -N '""'
```

5. Installer la clé **publique** sur le serveur. Pour un compte administrateur, Windows lit `C:\ProgramData\ssh\administrators_authorized_keys` (et non le dossier du profil) :

```
# Sur le serveur, en administrateur, coller le contenu de deploy-ga-151.pub :
Add-Content C:\ProgramData\ssh\administrators_authorized_keys "<contenu de deploy-ga-151.pub>"
icacls C:\ProgramData\ssh\administrators_authorized_keys /inheritance:r /grant "Administrators:F"
"SYSTEM:F"
```

6. Tester depuis le PC : `ssh -i $env:USERPROFILE\.ssh\deploy-ga-151 deploy-ga@151.80.218.41 hostname` doit afficher le nom du serveur, sans mot de passe.
7. Jenkins > Credentials > **Add** > *SSH Username with private key* : ID `srv151-ssh`, utilisateur `deploy-ga`, clé privée = contenu de `deploy-ga-151`. Supprimez ensuite le fichier de clé privée du profil si vous ne l'utilisez plus à la main.
8. Administrer Jenkins > System > Global properties > Environment variables :

Nom	Valeur
`MODE_DEPLOIEMENT`	`ssh`
`SERVEUR_151`	`151.80.218.41`
`URL_PUBLIQUE`	`http://151.80.218.41`

9. Lancer un build de `master` : Jenkins envoie `livraison-N.zip` et `deployer.ps1` dans `C:\GestionAgenda\livraisons` par `scp`, puis exécute `deployer.ps1` par `ssh`. Le pare-feu du bureau (FortiGate) doit autoriser le port 22 sortant vers 151.80.218.41.

3.4 Options utiles

- **Validation manuelle avant la production** : ajouter au début de l'étape de déploiement du `Jenkinsfile` `input message: 'Déployer en production ?', ok: 'Déployer'` ; Jenkins attend un clic.
- **Notifications** : plugin *Email Extension*, bloc `post { failure { emailext ... } }` pour prévenir l'équipe en cas d'échec.
- **Fenêtre de déploiement** : le déploiement coupe l'API environ 10 à 30 secondes (arrêt et redémarrage du pool). Pour éviter les heures ouvrées, fusionner sur `master` en fin de journée ou utiliser la validation manuelle.

3.5 Ce qui se passe à chaque fusion sur `master`

1. GitHub notifie Jenkins (cas A) ou Jenkins détecte le commit (cas B).
2. Compilation Release, tests .NET, tests Angular : un échec arrête tout, rien n'est déployé.
3. Construction de `livraison-N.zip` (archivée dans Jenkins : on peut la réinstaller à la main plus tard).
4. `deployer.ps1` sur le serveur : sauvegarde, migrations, copie, redémarrage, contrôle de santé.
5. Vérification publique : `http://151.80.218.41` doit répondre 200.
6. Build vert : la nouvelle version est en ligne. Build rouge : lire le journal de l'étape en échec (annexe A).

Annexe A — Dépannage

Symptôme	Cause probable	Solution
HTTP 500.19 sur l'API ou le front	Hosting Bundle ou URL Rewrite absent	Installer (2.1), puis `iisreset`
HTTP 500.30 / 502.5 sur l'API	L'API plante au démarrage : `Jwt:Secret` absent ou trop court, base inaccessible, JSON invalide	Observateur d'événements > Application. Pour plus de détails, mettre temporairement `stdoutLogEnabled="true"` dans `C:\GestionAgenda\api\web.config` et lire `C:\GestionAgenda\api\logs`
« Login failed for user IIS APPPOOL\GestionAgenda-API »	Login SQL non créé	Étape 2.4
Erreur CORS dans la console du navigateur	`Cors:Origins` ne contient pas l'adresse exacte du front	Corriger `appsettings.Production.json` (schéma, hôte, port, sans `/` final), redémarrer le pool
Page blanche, requêtes vers `/api` en 404	`assets\config.json` absent ou faux	Étape 2.6
Rafraîchir une page Angular donne 404	URL Rewrite absent	Étape 2.1
Pas de notifications en temps réel	WebSockets IIS non installé, ou port 8081 filtré	`Install-WindowsFeature Web-WebSockets`, pare-feu
Rappels ou synchronisations Google qui ne partent plus	Pool arrêté par inactivité	Vérifier `AlwaysRunning`, `idleTimeout = 0`, `preloadEnabled` (2.3)
« Hôte inconnu (oauth2.googleapis.com) »	Sortie Internet ou DNS bloqué	Autoriser `*.googleapis.com` en sortie ; Hangfire relance automatiquement
Connexion impossible avec un compte créé en SQL	E-mail non mis en minuscules ou hachage incorrect	Utiliser `LOWER(...)` et `hash-mot-de-passe.cs`
Jenkins : « dotnet / npm n'est pas reconnu »	Outils installés après Jenkins	Redémarrer le service Jenkins
Jenkins : « UNPROTECTED PRIVATE KEY FILE »	Droits du fichier de clé temporaire	Le `Jenkinsfile` corrige les droits avec `icacls` ; vérifier que le service tourne sous `svc-jenkins` et non `SYSTEM`
`deployer.ps1` : « Le pool ne s'arrête pas »	Requête longue en cours	Relancer le déploiement ; sinon `iisreset /stop`, déployer, `iisreset /start`
`deployer.ps1` : migrations en échec	Script SQL incompatible avec l'état de la base	La sauvegarde `.bak` indiquée dans le message permet de restaurer ; corriger la migration puis relivrer

Annexe B — Liste de contrôle de mise en production

- [ ] Cache `.angular` retiré de Git, aucun secret dans le dépôt ni l'historique
- [ ] Dépôt GitHub privé, `master` protégée
- [ ] Serveur : IIS + WebSockets + Application Initialization, Hosting Bundle .NET 10, URL Rewrite, SQL Server
- [ ] Fuseau horaire du serveur réglé
- [ ] Sites `GestionAgenda-Front` (80) et `GestionAgenda-API` (8081), pool de l'API `AlwaysRunning`
- [ ] Base `GestionAgenda` créée, login du pool `db_owner`
- [ ] `appsettings.Production.json` rempli et protégé : connexion, **nouveau** `Jwt:Secret`, CORS, SMTP, Google, `Seed:Enabled = false`
- [ ] `assets\config.json` pointe sur `http://151.80.218.41:8081/api`
- [ ] **Nouvelle** clé Google de production copiée dans `C:\GestionAgenda\secrets`, agendas et Sheet partagés avec le compte de service
- [ ] Première livraison installée, comptes initiaux créés, vérifications 2.10 passées
- [ ] Jenkins : identifiants, job multibranche, webhook ou scrutation, `MODE_DEPLOIEMENT` défini
- [ ] Un déploiement automatique complet réussi depuis `master`

Annexe C — Passer en HTTPS avec un nom de domaine

1. Créer deux enregistrements DNS de type A vers `151.80.218.41`, par exemple `agenda.<domaine>` (front) et `api-agenda.<domaine>` (API).
2. Installer **win-acme** (`wacs.exe`) sur le serveur. Il obtient des certificats Let's Encrypt gratuits, crée les liaisons HTTPS (443) dans IIS et les renouvelle automatiquement.
3. Dans IIS, ajouter à chaque site une liaison `https`, port 443, avec le nom d'hôte correspondant (SNI). L'API peut alors quitter le port 8081.
4. Mettre à jour les adresses :
   - `Cors:Origins` : `https://agenda.<domaine>`
   - `GoogleCalendar:UrlApplication` : `https://agenda.<domaine>`
   - `assets\config.json` : `{"apiUrl": "https://api-agenda.<domaine>/api"}`
   - `deployer.ps1` : paramètre `-UrlSante` si l'API n'écoute plus sur `http://localhost:8081`
   - Jenkins : `URL_PUBLIQUE` = `https://agenda.<domaine>`
5. Les notifications push du navigateur deviennent disponibles (clés VAPID renseignées).